03_data-overview

July 8, 2026

[ ]:
```
# %%
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from consts import *
import numpy as np
from itertools import combinations
from scipy.stats import linregress
from scipy.stats import spearmanr
from scipy.stats import pearsonr
import json
from matplotlib.patches import Patch


bigfs = 11
smallfs = 9
tinyfs = 7
plt.rcParams.update({"font.size":smallfs})
plt.rcParams.update({"figure.figsize":(16/2.54, 9/2.54)})
sns.set_style("ticks")
sns.set_context("paper")
```

1 Variable Overview

Load Data

[ ]:
```
df_orig =  pd.read_csv("processed_data/2026-05-13_allBilendiData.csv")
```

[ ]:
```
# %%
df_p = pd.read_csv("processed_data/2026-06-19_data_processed_participant.csv")
print("Full Size of participant data: ", df_p.wave.value_counts().to_dict())

inds_bothwaves = df_p["id"].value_counts().reset_index().
 ↪query("count==2")["id"].tolist()
df_p_bothwaves = pd.read_csv("processed_data/
 ↪2026-06-19_data_processed_participant_pivot.csv").query(f"id in␣
 ↪{inds_bothwaves}")
```

```
print("Full Size of across-wave data: ", len(df_p_bothwaves))
```

[ ]:
```
# %%
df_diff = pd.read_csv("processed_data/2026-06-19_data_processed_differences.
 ↪csv")
# df_diff = df_diff.loc[((df_diff["id"].isin(ids_w1)) & (df_diff["wave"]==1)) |␣
 ↪((df_diff["id"].isin(ids_w2)) & (df_diff["wave"]==2))]
print("Full Size of pairwise data (in wave 1 and wave 2): ", df_diff["wave"].
 ↪value_counts().to_dict())
print(df_diff.shape)
print(len(df_diff.loc[df_diff.wave==1,"id"].unique()))
print(len(df_diff.loc[df_diff.wave==2,"id"].unique()))
# print(df_diff.columns)


cols_to_keep = ["id", "wave"] + [f"std_socialCircle_ops_{q}" for q in␣
 ↪questions_sc]
cols_to_keep +=  ["average_pixel_dist", "average_pixel_dist_parties"]
df_diff = df_diff.merge(df_p[cols_to_keep], on=["id", "wave"], how="left")
df_diff["rel_pixel_dist"] = df_diff["pixel_dist"]/df_diff["average_pixel_dist"]
df_diff["rel_pixel_distP"] = df_diff["pixel_dist"]/
 ↪df_diff["average_pixel_dist_parties"]
```

1.1 Time Analyses

[ ]:
```
# %%
plt.figure(figsize=(12/2.54, 5/2.54))
sns.histplot((pd.to_datetime(df_p_bothwaves["wave2_t_completed"]) - pd.
 ↪to_datetime(df_p_bothwaves['wave1_t_completed'])).dt.days, bins=np.arange(0.
 ↪5,100.5))
plt.xlabel("time between wave 1 and wave 2 [days]" )
```

[ ]:
```
# %%
fig, ax = plt.subplots(1,1, sharex=False, figsize=(3,2))
t = 'time_total'
vmax = np.percentile(df_p.assign(t_div_60=df_p[t] / 60)['t_div_60'].values, 95)␣
 ↪* 3
df_p = df_p.copy()
sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave", bins␣
 ↪= np.arange(0,vmax,3),  alpha=0.5, palette=cmapWave, hue_order=[2,1])
ax.set_title(t)
ax.text(0.95,0.95,
        "\n".join([f"{np.sum(df_p.loc[df_p.wave==w, t].values / 60 < vmax)/df_p.
 ↪loc[df_p.wave==w, t].count()*100:.1f}% with <{vmax:.1f}min (wave {w})" for w␣
 ↪in [1,2]]),
        ha="right", va="top", transform=ax.transAxes, fontsize=7)
```

```
ax.set_xlabel("time in min")
fig.tight_layout()

print(f"median ({t}): {df_p[t].div(60).median():.2f} (25%-perc: {df_p[t].
 ↪div(60).describe()['25%']:.2f}, 75%-perc: {df_p[t].div(60).describe()['75%']:
 ↪.2f})")

times = ['time_trainingGame', 'time_training', 'time_spam', 'time_spam18dots',␣
 ↪'time_pairwise', 'time_pairwise18pairs']
fig, axs = plt.subplots(2,3, sharex=False, figsize=(7,3))
for ax, t in zip(axs.flatten(), times):
    vmax = np.percentile(df_p[t].dropna().values, 95) * 3 / 60
    df_p = df_p.copy()
    sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",␣
 ↪bins = np.arange(0,vmax,0.5),  alpha=0.5, ax=ax, legend=False,␣
 ↪palette=cmapWave, hue_order=[2,1])
    ax.set_title(t)
    ax.text(0.95,0.95,
            "\n".join([f"{np.sum(df_p.loc[df_p.wave==w, t].values / 60 < vmax)/
 ↪df_p.loc[df_p.wave==w, t].count()*100:.1f}% with <{vmax:.1f}min (wave {w})"␣
 ↪for w in [1,2]]),
            ha="right", va="top", transform=ax.transAxes, fontsize=7)
    ax.set_xlabel("time in min")
fig.tight_layout()

display(df_p[times].div(60).describe())
display(df_p[times].describe())


# for t in [4,6]:
#     descr = df_p[time_cols(1)[t]].div(60).describe(percentiles=[0.05,0.25,0.
 ↪5,0.75,0.95])
#     print(f"median ({time_cols(1)[t]}): {descr["50%"]:.2f} (25%-perc:␣
 ↪{descr["25%"]:.2f}, 75%-perc: {descr["75%"]:.2f}), (5%-perc: {descr["5%"]:.
 ↪2f}, 95%-perc: {descr["95%"]:.2f})")
#     descr = df_p[time_cols(2)[t]].div(60).describe(percentiles=[0.05,0.25,0.
 ↪5,0.75,0.95])
#     print(f"median ({time_cols(2)[t]}): {descr["50%"]:.2f} (25%-perc:␣
 ↪{descr["25%"]:.2f}, 75%-perc: {descr["75%"]:.2f})  (5%-perc: {descr["5%"]:.
 ↪2f}, 95%-perc: {descr["95%"]:.2f})")
```

Note: some of the times are negative and should not be negative. This is probably because participants re-loaded the page and reloading overwrites the visited time stamp in the dataset.

1.2 Demographics

1.2.1 Age Gender Region (bilendi meta-data)

[ ]:
```
wavecondition = "wave in [1]"
print("Age: ", df_p.query(wavecondition)["age"].describe())
print("Gender: ", (df_p.query(wavecondition)["gender"].value_counts().
 ↪sort_values(ascending=False)/df_p.query("wave==1")["region"].count()).
 ↪to_dict())
print("Region: ", (df_p.query(wavecondition)["region"].value_counts().
 ↪sort_values(ascending=False)/df_p.query("wave==1")["region"].count()).
 ↪to_dict())
```

[ ]:
```
# %%
demo_cols = ["gender", "age", "party_vote", "region"]  # participant-level␣
 ↪constants
for d in demo_cols:
    fig = plt.figure(figsize=(4,1.5))
    ax = plt.axes()
    sns.histplot(df_p[d], ax=ax, color=cmapWave[1])
    fig.autofmt_xdate()
```

1.2.2 Parties / political identity

[ ]:
```
wavecondition = "wave in [1]"
partyPrefsVote = df_p.query(wavecondition)["party_vote"].value_counts().
 ↪sort_values(ascending=False)
partyPrefsClose = df_p.query(wavecondition)["party_close"].value_counts().
 ↪sort_values(ascending=False)
partyPrefs = (
    partyPrefsVote.to_frame("vote")
    .join(partyPrefsClose.to_frame("close"), how="outer")
    .fillna(0).astype(int)
    .sort_values("vote", ascending=False)
)
print("Party Vote/Close: ", partyPrefs)
print("Party Vote/Close Fraction: ",partyPrefs/ partyPrefs.sum(axis=0))
```

[ ]:
```
# %%
party_cmap["Miscellaneous"] = "brown"
party_cmap["Not Voting"] = "darkgrey"
fig, axs = plt.subplots(3,1, sharex=True)
for d, w, name, ax in zip(["party_vote", "party_close", "party_close"],␣
 ↪[1,1,2], ["vote", "close_wave1", "close_wave2"], axs.flatten()):
    sns.barplot(df_p.loc[df_p.wave==w, d].value_counts()[parties_vote if "vote"␣
 ↪in d else parties_full].reset_index(), ax=ax, y="count", x=d, hue=d,␣
 ↪palette=party_cmap)
```

```
    fig.autofmt_xdate()
    ax.set_ylabel(name)
    ax.set_xlabel("")
```

[ ]:
```
# %%
fig, axs = plt.subplots(3,1, figsize=(14/2.54, 12/2.54), sharex=True,␣
 ↪sharey=False)
for (d1, d2), ax in zip(combinations(["party_vote", "wave1_party_close",␣
 ↪"wave2_party_close"], 2), axs):

    sns.heatmap(df_p_bothwaves[[d1,d2,"age"]].pivot_table(index=d1,columns=d2,␣
 ↪aggfunc="count",values="age").loc[parties_full if "wave" in d1 else␣
 ↪parties_vote, parties_full if "wave" in d2 else parties_vote], cmap="Reds",␣
 ↪ax=ax, cbar=False,)
    # ax.set_aspect("equal")
    ax.set_title(d2)
    # ax.set_ylabel("")
axs[-1].set_xlabel("")
fig.autofmt_xdate(rotation=30)
fig.tight_layout()
```

[ ]:
```
# %%
for var in ["lr", "polInterest", "polFrequency", "n_contacts"]:
    fig, axs = plt.subplots(1,2, figsize=(14/2.54,5/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,␣
 ↪palette=cmapWave, hue_order=[2,1], bins=11)
    sns.regplot(df_p_bothwaves[["wave2_"+var, "wave1_"+var]], x="wave1_"+var,␣
 ↪y="wave2_"+var, ax=axs[1], scatter_kws={"s":3, "alpha":0.5, "color":"grey"},␣
 ↪y_jitter=0.2*(var=="n_contacts"), x_jitter=0.2*(var=="n_contacts"))
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].corr().values[1,0]}")
```

1.3 Opinions

opinion change within survey

[ ]:
```
# %%
# Changes in opinion within the survey wave on slide 1 and slide 5(?)

vars = [f"x_self_{q}" for q in questions_sc]
varsPrior = [f"first_x_self_{q}" for q in questions_sc]
wave = 2
for var, varp, q in zip(vars, varsPrior, questions_sc):
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p[[varp, var]], ax=axs[0], alpha=0.6)
    sns.regplot(df_p, x=varp, y=var, ax=axs[1], scatter_kws={"s":3, "alpha":0.
 ↪5, "color":"grey"})
```

```
    axs[1].set_aspect("equal")
    print(f"correlation wave {wave} {var} and {varp}: {df_p[[varp, var]].corr().
 ↪values[1,0]}")
    ax.set_title(q)
    fig.tight_layout()


print(f"number of people who changed their opinions: {dict(zip(questions_sc,␣
 ↪((np.abs(df_p[[var for  var in vars]].values - df_p[[varp for  varp in␣
 ↪varsPrior]].values)>0).sum(axis=0))))}")
```

1.3.1 Own Opinion Distributions

[ ]:
```
# %%
vars = [f"x_self_{q}" for q in questions_sc]
for var, q in zip(vars, questions_sc):
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p.loc[df_p.id.isin(df_p_bothwaves.id)], x =␣
 ↪f"x_self_{q}",ax=axs[0], hue="wave", bins=11, binrange=(-1,1), alpha=0.6, ␣
 ↪legend=False, stat="percent", multiple="layer", kde=False, palette=cmapWave,␣
 ↪hue_order=[2,1])
    sns.regplot(df_p_bothwaves[[f"wave{w}_"+var for w in [1,2]]],␣
 ↪x="wave1_"+var, y="wave2_"+var, ax=axs[1], scatter_kws={"s":3, "alpha":0.5,␣
 ↪"color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

# display(df_p[[var for  var in vars]].describe())
```

[ ]:
```
wavecondition = "wave in [1]"

op_cols = [f"x_self_{q}" for q in questions_sc]
fig, axes = plt.subplots(2, 3, figsize=(7, 3.5), sharey=True, sharex=True)
axes = axes.flatten()

for ax, col, q in zip(axes, op_cols, questions_sc):
    sns.histplot(
        df_p.query(wavecondition)[col].dropna(),
        ax=ax,
        bins=11,
        binrange=(-1.01, 1.01),
        # kde=True,
        stat="density",      # makes KDE and bars scale together properly
        color=cmapQuestions[q],
        alpha=0.5,
```

```
    )
    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
 ↪edgecolor='none', pad=4), fontsize=bigfs)
    ax.set_xlim(-1, 1)
    ax.set_yticks([])
    ax.set_ylabel("")
    ax.set_xlabel("")
    ax.tick_params(axis="x")

axes[-2].set_xlabel("own opinions", fontsize=bigfs)
fig.tight_layout()
plt.savefig("figs/ownOps.png")
```

1.3.2 Opinions of Voters

[ ]:
```
# %%
fig, axs = plt.subplots(2,3, figsize=(16/2.54,8/2.54), sharex=True, sharey=True)
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"x_{p}_{q}" for p in partiesVars]].melt(var_name="party", )
    a["party"]= a["party"].apply(lambda x: x.split("_")[1])
    sns.histplot(a, x="value", hue="party", palette=party_cmap,␣
 ↪hue_order=partiesVars, bins=11, binrange=(-1,1), ax=ax, alpha=0.6,␣
 ↪legend=False, stat="percent", multiple="layer", kde=False)
    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
 ↪edgecolor='none', pad=4), fontsize=bigfs)
    ax.set_xlabel("")
    ax.set_ylabel("")
    ax.set_yticks([])
fig.suptitle("What are the opinions of a typical voter of party...?")
fig.tight_layout()
```

[ ]:
```
fig, axs = plt.subplots(2, 3, figsize=(8, 3.5), sharey=False, sharex=True)
axes = axs.flatten()

for ax, q in zip(axes, questions_sc):
    op_cols_p = [f"x_{r}_{q}" for r in partiesVars]
    sns.kdeplot(
        df_p.query(wavecondition)[op_cols_p].dropna().melt().
 ↪replace(dict(zip(op_cols_p, partiesVars))),
        x="value",
        hue="variable",
        palette=party_cmap,
        ax=ax,
        fill=True,
        alpha=0.02,
        lw=2,
        legend=False,   # suppress all in-plot legends
```

```
    )

    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
 ↪edgecolor='none', pad=4), fontsize=bigfs)
    ax.set_xlim(-1, 1)
    ax.set_yticks([])
    ax.set_ylabel("")
    ax.set_xlabel("")
    ax.tick_params(axis="x")

axs[-1,1].set_xlabel("Perceived Opinion of Typical Voter", fontsize=bigfs)
# Build legend handles manually from the last axis
handles = [
    Patch(facecolor=party_cmap[p], alpha=0.4, label=p)
    for p in partiesVars
]
fig.legend(
    handles, partiesVars,
    loc="center left",
    bbox_to_anchor=(0.82, 0.5),   # just outside the right edge
    frameon=False,
)

fig.tight_layout(pad=0.6)
fig.subplots_adjust(right=0.82)  # make room for the legend
plt.savefig("figs/partyOps.png")
```

[ ]:
```
# %%
fig, axs = plt.subplots(2,2, figsize=(16/2.54,14/2.54), sharex=True,␣
 ↪sharey=True)
fig.suptitle("What are the opinions of a typical voter of party...?")
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"x_{p}_{q}" for p in partiesVars]].melt(var_name="party")
    a["party"]= a["party"].apply(lambda x: x.split("_")[1])
    sns.violinplot(a, x="value", y="party", hue="party", fill=False,␣
 ↪inner=None, palette=party_cmap, hue_order=partiesVars, ax=ax, legend=False,␣
 ↪cut=0, )
    sns.stripplot(a, x="value", y="party", hue="party", palette=party_cmap,␣
 ↪hue_order=partiesVars, ax=ax, legend=False, size=1)
    ax.set_title(q)
fig.tight_layout()
```

2 Std Dev of Opinions in social circles

[ ]:
```
wavecondition = "wave in [1]"
stdSC_cols = [f"std_socialCircle_ops_{q}" for q in questions_sc]
fig, axes = plt.subplots(2, 3, figsize=(7, 3.5), sharey=True, sharex=True)
axes = axes.flatten()
df_p["treatment_wave2_str"] = df_p["treatment_wave2"].map({1.0:"T", 0.0:"C", np.
 ↪nan: "w1"})
for ax, col, q in zip(axes, stdSC_cols, questions_sc):
    sns.histplot(
        df_p.query(wavecondition)[[col, "treatment_wave2_str"]].dropna(),
        x=col,
        ax=ax,
        bins=11,
        binrange=(0, 1.),
        kde=True,
        stat="density",      # makes KDE and bars scale together properly
        # color="orange",
        hue="treatment_wave2_str",
        palette={"T":"magenta", "C":"brown", "w1":"orange"},
        alpha=0.5,
        legend=False
    )
    mean_T = df_p.query(wavecondition+" and treatment_wave2_str=='T'")[col].
 ↪dropna().mean()
    mean_C = df_p.query(wavecondition+" and treatment_wave2_str=='C'")[col].
 ↪dropna().mean()
    mean_w1 = df_p.query(wavecondition+" and treatment_wave2_str=='w1'")[col].
 ↪dropna().mean()
    for meanval, col, y, label in zip([mean_T, mean_C, mean_w1], ["magenta",␣
 ↪"gold", "orange"], [0.8,0.65, 1], ["T", "C", "{w1}"]):
        ax.vlines(
            meanval,
            0,2.55,
            color=col if "2" in wavecondition else "darkorange",
            linestyles="--"
        )
        s = '{SD}'
        add = ""#f"_{label}"
        ax.text(meanval-0.05, 2.5, rf"   $\overline{s}{add}$:"+f" {meanval:.
 ↪2f}", ha='left', va='bottom', color=col)


    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
 ↪edgecolor='none', pad=4), fontsize=bigfs)
    ax.set_xlim(0, 1.)
    ax.set_yticks([])
```

```
    ax.set_ylabel("")
    ax.set_xlabel("")
    ax.tick_params(axis="x")
    # ax.text( df_p.query(wavecondition)[col].dropna().mean(), 2.8, f" mean:␣
 ↪{df_p.query(wavecondition)[col].dropna().mean():.2f}", ha='left', va='top')
axes[-2].set_xlabel("std opinions social circle")
print("social circle std")
fig.tight_layout()
plt.savefig("figs/socialvarOps.png")
```

[ ]:
```
# %%
fig, axs = plt.subplots(2,3, figsize=(16/2.54,14/2.54), sharex=True,␣
 ↪sharey=True)
fig.suptitle("Std Dev of Opinions in social circles")

for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"std_socialCircle_ops_{q}"]+[f"party_close"]].
 ↪melt(var_name="social circle", id_vars=[f"party_close"])
    sns.violinplot(a, x="value", y=f"party_close", hue=f"party_close",␣
 ↪fill=False,  palette=party_cmap, hue_order=parties_full, ax=ax,␣
 ↪legend=False, cut=0, inner="quart", order=parties_full)
    sns.stripplot(a, x="value", y=f"party_close", hue=f"party_close",␣
 ↪palette=party_cmap, hue_order=parties_full, ax=ax, legend=False, size=1)
    sns.stripplot(a.groupby(f"party_close")["value"].mean().reset_index(),␣
 ↪x="value", y=f"party_close", hue=f"party_close", palette=party_cmap,␣
 ↪hue_order=parties_full, ax=ax, legend=False, size=5, marker="s")
    ax.set_title(q)
    ax.tick_params(axis='y', labelsize=tinyfs)
    ax.set_ylabel("")
fig.tight_layout()
```

2.1 Treatment

[ ]:
```
# %%
print(f"Treatment: {df_p['treatment_wave2'].value_counts().to_dict()}")
```

[ ]:
```
# %%
fig, ax = plt.subplots(1,1, figsize=(16/2.54,7/2.54))
ax.set_title("Std Dev of Opinions in social circles")
a = df_p[[f"std_socialCircle_ops_{q}" for q in␣
 ↪questions_sc]+["treatment_wave2"]].melt(var_name="question",␣
 ↪id_vars=[f"treatment_wave2"])
a["question"]= a["question"].apply(lambda x: labelMap["_".join(x.split("_")[3:
 ↪])])
a = a.dropna()
```

```
sns.violinplot(a, x="value",  y="question",split=True, hue="treatment_wave2",␣
 ↪fill=False,   ax=ax, legend=False, cut=0, inner="quart",␣
 ↪palette=cmapTreatment, hue_order=[True,False])
sns.stripplot(a, x="value",  y="question",  hue=f"treatment_wave2",  ax=ax,␣
 ↪legend=False, size=0.6, dodge=True, jitter=0.3, palette=cmapTreatment,␣
 ↪hue_order=[True,False])
sns.stripplot(a.groupby([f"treatment_wave2", "question"])["value"].mean().
 ↪reset_index(), x="value", y=f"question", hue=f"treatment_wave2", ax=ax,␣
 ↪legend=True, size=5, marker="s", palette=cmapTreatment,␣
 ↪hue_order=[True,False], jitter=False)
ax.set_ylabel("")
fig.tight_layout()

fig, axs = plt.subplots(2,3, sharex=True, sharey=True, figsize=(16/2.54, 8/2.
 ↪54))
df_p["treatment_wave2"] = df_p["treatment_wave2"].astype(bool)
for ax, q in zip(axs.flatten(), questions_sc):
    sns.boxplot(df_p, x="treatment_wave2", y=f"std_socialCircle_ops_{q}", hue =␣
 ↪"treatment_wave2", ax=ax, fill=False, whis=[5,95], fliersize=0,␣
 ↪palette=cmapTreatment, hue_order=[1,0], legend=False)
    sns.stripplot(df_p, x="treatment_wave2", y=f"std_socialCircle_ops_{q}",␣
 ↪ax=ax, hue = "treatment_wave2", size=1,alpha=0.3, palette=cmapTreatment,␣
 ↪hue_order=[True,False], legend=False)
    sns.stripplot(df_p.groupby("treatment_wave2")[f"std_socialCircle_ops_{q}"].
 ↪mean().reset_index(), x="treatment_wave2", y=f"std_socialCircle_ops_{q}",␣
 ↪hue = "treatment_wave2", edgecolor="k", linewidth=1, ax=ax, size=4,␣
 ↪marker="s",alpha=1, palette=cmapTreatment, hue_order=[True,False],␣
 ↪legend=False)
    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
 ↪edgecolor='none', pad=4), fontsize=bigfs)
    ax.set_xlabel("")
    ax.set_ylabel("")
axs[0,0].set_ylabel(r"social circle opinion std $SD$", y=-0)
axs[-1,1].set_xlabel("Treatment")
fig.tight_layout()
plt.savefig("figs/SD_treatment.png", dpi=600)
```

2.2 Polarisation

[ ]:
```
# %%
for var in ["P_tot"]+[f"P_{q}" for q in questions_sc]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,␣
 ↪palette=cmapWave, hue_order=[2,1], bins=11)
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],␣
 ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
```

```
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].corr().values[1,0]}")
descr1P = df_p[["P_tot"]+[f"P_{q}" for  q in questions_sc]].describe()
display(descr1P)
```

[ ]:
```
# %%
descr1P.loc[["mean", "std", "50%"]].plot.bar(color=["k"]+list(cmapQuestions.
 ↪values()))
plt.ylabel("issue polarisation")
```

2.3 Issue Importance

[ ]:
```
# %%
for var in [f"w_{q}" for q in questions_sc]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,␣
 ↪palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],␣
 ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()
descr1w = df_p[[f"w_{q}" for  q in questions_sc]].describe()
display(descr1w)
```

[ ]:
```
# %%
descr1w.loc[["mean", "std", "50%"]].plot.bar(color=cmapQuestions.values())
plt.ylabel("issue importance")
```

[ ]:
```
cmapQuestions
```

[ ]:
```
# %%
df_p[[f"w_{q}" for  q in questions_sc]].describe()
```

[ ]:
```
# %%
var = "sum_issue_importance"
fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
df_p[var] = df_p[[f"w_{q}" for  q in questions_sc]].sum(axis=1)
sns.histplot(df_p,x=var, hue="wave", ax=axs[0], alpha=0.6, palette=cmapWave,␣
 ↪hue_order=[2,1])
aa = df_p.pivot_table(index="id", columns="wave", values=var).rename(columns={1:
 ↪"wave1", 2:"wave2"})
sns.regplot(aa, x="wave1", y="wave2", ax=axs[1], scatter_kws={"s":3, "alpha":0.
 ↪5, "color":"grey"})
axs[1].set_aspect("equal")
```

```
print(f"correlation wave 2 wave 1 {var}: {aa.corr().values[1,0]}")
fig.tight_layout()
display(df_p[var].describe())
```

2.4 Voter Sympathy/Likeability

[ ]:
```
# %%

N = len(partiesVars)
ncols = 2
nrows = (N + 1) // ncols  # ceiling division

fig, axs = plt.subplots(
    nrows, ncols,
    figsize=(16/2.54, 12/2.54),
    sharex=True, sharey=True
)
axs_flat = axs.flatten()

for i, p in enumerate(partiesVars):
    ax = axs_flat[i]
    sns.histplot(
        data=df_p,
        x=f"sym_{p}",
        hue="party_close",
        ax=ax,
        alpha=0.6,
        palette=party_cmap,
        multiple="stack",
        bins=np.linspace(-0.01, 1.01, 21),
        legend=(i == N - 1),
        linewidth=0,
        hue_order=parties_full,
    )
    ax.set_title(f"towards {p} voter", x=0.95, y=0.8,
                 color=party_cmap[p], va="top", ha="right")
    ax.set_ylabel("")
    ax.set_xlabel("sympathy towards typical voter" if i >= N - ncols else "")

# Hide unused axes (last cell if N is odd)
for ax in axs_flat[N:]:
    ax.axis("off")

ax = axs_flat[N-1]
sns_legend = ax.get_legend()
handles = sns_legend.legend_handles
labels = [t.get_text() for t in sns_legend.get_texts()]
```

```
sns_legend.remove()

# Hide the last axes but keep it as a legend host
empty_ax = axs_flat[N]
empty_ax.axis("off")

empty_ax.legend(
    handles, labels,
    bbox_to_anchor=(0.05, 0.5),
    ncol=2,
    fontsize=6,
    title="party id",
    title_fontsize=6,
    loc="center left",
    borderaxespad=0,
)

display(df_p[[f"sym_{p}" for p in partiesVars]].describe())

fig.tight_layout()
plt.savefig("sympathy.png", dpi=600)
```

3 Traning

[ ]:
```
var = "attemptsPractice"
print(f"Fraction of training fails in wave 1: {sum((df_p.
 ↪wave==1)*(df_p[var]==-999))/sum(df_p.wave==1):.4f}")
# df_p[["wave", var]].value_counts().reset_index().sort_values("wave")
```

[ ]:
```
# %%
for var in ["attemptsPractice"]:
    fig, ax = plt.subplots(1,1, figsize=(3,2))
    sns.histplot(df_p.replace({-999:7}), x=var, hue="wave", ax=ax,␣
 ↪palette=cmapWave, bins=np.array([[k-0.35, k+0.35] for k in range(1,8)]).
 ↪flatten(), alpha=0.6, multiple="dodge", legend=False, stat="count")
    ax.set_xticks([1,2,3,4,5, 7])
    ax.set_yticks([])
    ax.set_xticklabels(["1","2","3","4","5", "failed"])
    ax.set_xlabel("nr of attempts to pass training")
    # wave 1 -> left sub-bar of the "failed" group
    ymax = ax.get_ylim()[1]
    # wave 1 -> left sub-bar (shorter elbow, drops in closer to the label)
    ax.annotate(
        "wave 1",
        xy=(6.8, df_p.loc[df_p.wave==1, var].value_counts()[-999]),␣
 ↪xycoords="data",           # arrow tip on left bar
```

```
        xytext=(6.2, ymax * 0.9), textcoords="data",
        fontsize=8, ha="right", va="center",
        arrowprops=dict(
            arrowstyle="->", color="black", lw=1,
            connectionstyle="angle,angleA=0,angleB=90,rad=0"
        )
    )

    # wave 2 -> right sub-bar (longer horizontal run, sits above wave 1's label)
    ax.annotate(
        "wave 2",
        xy=(7.2, df_p.loc[df_p.wave==2, var].value_counts()[-999]),␣
 ↪xycoords="data",              # arrow tip on right bar
        xytext=(6.2, ymax * 1), textcoords="data",
        fontsize=8, ha="right", va="center",
        arrowprops=dict(
            arrowstyle="->", color="black", lw=1,
            connectionstyle="angle,angleA=0,angleB=90,rad=0"
        )
    )
    ax.set_ylim(top=ymax * 1.1)  # give headroom for the labels
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].replace({-999:np.nan}).corr().values[1,0]}")
    plt.tight_layout()
    plt.savefig("figs/trainingattempts.png", dpi=600 )
```

[ ]:
```
# %%
for var in ["attemptsPractice"]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))

    sns.histplot(df_p.replace({-999:8}), x=var, hue="wave", ax=axs[0], bins=0.
 ↪5+np.arange(0,9), alpha=0.6, palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves[["wave1_"+var, "wave2_"+var]].replace({-999:np.
 ↪nan}), x="wave1_"+var, y="wave2_"+var, ax=axs[1], scatter_kws={"s":3,␣
 ↪"alpha":0.5, "color":"grey"}, y_jitter=0.2*(var=="attemptsPractice"),␣
 ↪x_jitter=0.2*(var=="attemptsPractice"))
    axs[1].set_aspect("equal")

    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].replace({-999:np.nan}).corr().values[1,0]}")
```

[ ]:
```
# %%
display(df_p[[f"dist_game_{a[0]}-{b[0]}" for a,b in␣
 ↪combinations(practice_game_dots, 2)]].describe())
fig = plt.figure(figsize=(2,2))
df_p[[f"dist_game_{a[0]}-{b[0]}" for a,b in combinations(practice_game_dots,␣
 ↪2)]].mean().plot.bar()
```

```
fig.autofmt_xdate()
```

[ ]:
```
# %%
vars = [f"dist_practice_{a[0]}-{b[0]}" for a,b in␣
 ↪combinations(practice_training_dots, 2)]
display(df_p[vars].describe())
fig = plt.figure(figsize=(6,2))
df_p[vars].mean().plot.bar(color=plt.get_cmap("tab10").colors)
plt.xticks(plt.gca().get_xticks(), labels=[s[-3:]  for s in vars])
plt.title("Distances between dots in Training (self s, friend f, co-worker c,␣
 ↪relative r)")
fig.autofmt_xdate()
```

[ ]:
```
# %%
df_p[["wave", "passed_practice_sanity"]].replace({np.nan:"failed training"}).
 ↪groupby("wave")["passed_practice_sanity"].value_counts().reset_index()
```

4 Mapping

[ ]:
```
wave = 2
ids = df_p.loc[df_p.wave==wave]["id"].sample(9).values

def plot_map(ax, x, q, pos_processed, wave):
        colors = colorsOrig if q=="" else [x["player."+f"{'own2' if p=='self'␣
 ↪else p}__{q}"].values[wave-1] for p in peeps]
        if q=="":
            ax.scatter([pos_processed[0,0]],[pos_processed[0,1]], c=colors[0],␣
 ↪s=20, marker="X")
            ax.scatter(pos_processed[1:,0],pos_processed[1:,1], c=colors[1:],␣
 ↪s=20, )
        else:
            ax.scatter([pos_processed[0,0]],[pos_processed[0,1]],␣
 ↪c=colors[0],s=20, cmap=plt.get_cmap("coolwarm"), vmin=-100,vmax=100,␣
 ↪marker="X")
            ax.scatter(pos_processed[1:,0],pos_processed[1:,1], c=colors[1:],␣
 ↪s=20, cmap=plt.get_cmap("coolwarm"), vmin=-100,vmax=100, )
        ax.set_aspect("equal")
        ax.set_xlim(0,MAX_PIXELPOS)
        ax.set_ylim(0,MAX_PIXELPOS)
        ax.set_xticks([])
        ax.set_yticks([])

fig, axs = plt.subplots(3,3, figsize=(12/2.54, 12/2.54), sharex=True,␣
 ↪sharey=True)
for id, ax in zip(ids, axs.flatten()):
    partic_data = df_orig.loc[(df_orig.wave==wave) & (df_orig.bilendi_id==id)]
```

```
    pos = json.loads(partic_data["player.positions"].values[0])
    pos_processed = {p["varname"].replace(" ",""):np.array([p["x"], p["y"]])␣
↪for p in pos}
    pos_processed = np.array([[np.nan, np.nan] if k.replace(" ", "") not in␣
↪pos_processed else pos_processed[k] for k in peeps])
    # pos_processed_dict = {k: [np.nan, np.nan] if k not in pos_processed else␣
↪pos_processed[k] for k in peeps}

    plot_map(ax, partic_data, "", pos_processed, np.nan)
    ax.text(0.0,1.02,f"id: {id} (w{wave})", transform=ax.transAxes, fontsize=5)
fig.tight_layout(h_pad=1, w_pad=1)
plt.show()
```

[ ]:
```
# %%
vars = ["mappingEnjoy", "mappingEasier", "map_satisfaction"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p,x=var, hue="wave", ax=axs[0], alpha=0.6,␣
↪palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],␣
↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
fig, axs = plt.subplots(1,3, figsize=(16/2.54,6/2.54))
for (var1, var2), ax in zip(combinations(vars,2), axs.flatten()):
    sns.regplot(df_p, x=var1, y=var2, ax=ax, scatter_kws={"s":3, "alpha":0.5,␣
↪"color":"grey"})
fig.tight_layout()
```

[ ]:
```
fig, axes = plt.subplots(1,3, figsize=(7, 2.), sharey=False, sharex=False)
vars = ["mappingEnjoy", "mappingEasier", "map_satisfaction"]
colors = dict(zip(vars, ["green", "olive", "limegreen"]))

for ax, col in zip(axes, vars):
    sns.histplot(df_p.query(f"wave==1")[col], color=colors[col], bins=11,␣
↪binrange=[-0.01,1.01] if "satis" in col else [-1.01,1.01], ax=ax, kde=True,␣
↪linewidth=3, alpha=0.3)
    ax.set_yticks([])
    ax.set_ylabel("responses" if "sim" in col else "")
```

```
    ax.set_xlabel(r"I enjoyed the mapping task more"+"\nthan the pairwise task"␣
 ↪if "Enjoy" in col else (r"I found the mapping task easier"+"\nthan the␣
 ↪pairwise task" if "Easier" in col else r"I am satisfied with the map I␣
 ↪created"+"\n"))
plt.tight_layout()
plt.savefig("figs/mappingTaskEvaluation.png", dpi=600)
```

[ ]:
```
# %%
vars = ["average_pixel_dist", "average_pixel_dist_parties"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,␣
 ↪palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],␣
 ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
```

[ ]:
```
# %%
vars = ["average_pixel_dist", "average_pixel_dist_parties"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p.loc[df_p["wave"]==2], x=var, hue="treatment_wave2",␣
 ↪ax=axs[0], alpha=0.6, palette=cmapTreatment)
    for t in [0,1]:
        sns.regplot(df_p_bothwaves.loc[df_p_bothwaves.
 ↪wave2_treatment_wave2==t], x="wave1_"+var, y="wave2_"+var, ax=axs[1],␣
 ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"}, color=cmapTreatment[t])
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,␣
 ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
```

5 Social Closeness

[ ]:
```
fig, axs = plt.subplots(1,4, figsize=(18/2.54, 5/2.54))
df_diff["dottype"] = df_diff.apply(lambda x: "personal" if (("reference" in␣
 ↪x['dot1'] or "self" in x["dot1"]) and ("reference" in x['dot2'] or "self" in␣
 ↪x["dot2"])) else "voter", axis=1)
```

```
hue= None #"dottype"
hue_order = None #["voter", "personal"] #[2,1]
cmap =  "#1f78b4" # {"voter":"k", "personal":party_cmap["contact"]}#
mult = "stack"
sns.histplot(df_diff, x="pairwise_similarity", hue=hue, palette=cmap,␣
 ↪color=cmap, hue_order=hue_order, ax=axs[0], multiple=mult, kde=True, bins=21)
axs[0].set_xlabel("pairwise similarity")

sns.histplot(df_diff, x="pixel_dist", hue=hue, palette=cmap,␣
 ↪color=cmap,hue_order=hue_order, ax=axs[1], multiple=mult, kde=True, bins=21)
axs[1].set_xlabel("map distance")

sns.histplot(df_diff, x="sympathy", hue=hue, palette=cmap,␣
 ↪color=cmap,hue_order=hue_order, ax=axs[2], multiple=mult, kde=True, bins=21)
axs[2].set_xlabel("likability [voters]")


sns.histplot(df_diff, x="socialCloseness", hue=hue, palette=cmap,␣
 ↪color=cmap,hue_order=hue_order, ax=axs[3], multiple=mult, kde=True, bins=21)
axs[3].set_xlabel("social closness [contacts]")

for ax in axs.flatten():
    ax.set_ylabel("")
    ax.set_yticks([])


fig.tight_layout()
```

[ ]:
```
df_diff.socialCloseness
```